

p0518r0

Allowing copies as arguments to function objects given to parallel algorithms in response to CH11

Published Proposal, 31 January 2017

This version:

<http://wg21.link/p0518>

Authors:

[David S. Hollman](#) (Sandia National Labs)

[Jonathan Lifflander](#) (Sandia National Labs)

[Michael Wong](#) (Codeplay)

[Detlef Vollmann](#) (Vollmann Engineering)

Audience:

SG1

Toggle Diffs:

☐ Hide deleted text

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

This paper addresses NB comment CH 11 in [\[P0488R0\]](#)

Table of Contents

- 1 **Background**
- 2 **Proposed Wording**
- 3 **Discussion and Justification**
 - 3.1 The "Default Function Object" Clause
- 4 **Summary**
- References**
 - Informative References

§ 1. Background

National Body comment CH11 from [\[P0488R0\]](#) states:

Comments: It may be useful to copy objects to a separate space for non-sequenced policies.

Proposed Change: Add explicit allowance for non-sequenced policies to copy the objects they work on.

Discussion in SG1 at Issaquah on this comment led to the suggestion that, at minimum, wording should be added to some or all of the parallel algorithms allowing the implementation to make copies of arguments to the function objects passed to a *parallel algorithm* [algorithms.parallel.defns]. A straw poll revealed unanimous agreement that a paper exploring this idea should be written.

In particular, it was suggested that we examine the consequences of expanding or elaborating on the clause in [algorithms.parallel.user] to include language allowing the implementation to make copies of arguments to certain function objects under certain circumstances.

§ 2. Proposed Wording

With discussion to follow below (§3 Discussion and Justification), we recommend that the committee change paragraph 1 of [algorithms.parallel.user] to:

Unless otherwise specified, function objects passed into parallel algorithms as objects of type `Predicate`, `BinaryPredicate`, `Compare`, `UnaryOperation` and `BinaryOperation` and the operators used by the analogous overloads to these parallel algorithms that could be formed by the invocation with the specified default predicate or operation (where applicable) shall not directly or indirectly modify objects via their arguments, which may be a copy of the original object(s) from the range(s) on which the algorithm operates. Copy operations for this purpose may only be performed on trivially copyable types ([basic.types]).

[Note: Programmers for whom the object identity of the arguments to these function objects is important should consider using a wrapping iterator that returns a noncopied implementation object such as `reference_wrapper<T>` [refwrap] or some equivalent solution. -end note]

§ 3. Discussion and Justification

For just about all of the algorithms, the primary concern is that a predicate or comparison operator would rely on some property of the arguments' addresses being consistent, whether in an absolute sense or a relative sense. (The more general phrasing for this concept is "object identity," but the most relevant aspect of object identity in the current context is the object's address, so the two terms will be used interchangeably here.) It is important to note that in every case, the parallel algorithm *could* still be used with a predicate reliant on the address if a layer of indirection that preserves the pertinent portion of the object's address information (e.g., through a wrapping iterator). Thus, it is not the actual breadth of capabilities provided by the parallel algorithm specifications that is being considered here; rather, it is the trade-offs between the cost of requiring the user to implement such a layer of indirection and the benefits of implementation flexibility with greater accessibility to performance optimizations.

In most cases, the algorithms relevant to the wording in §2 Proposed Wording that require the same type of function object pose similar concerns to each other. Of greatest concern are those functions accepting `Predicate` function objects. A typical case is `find_if` ([alg.find]), where the function object could want to check for equality with a known object by doing address-wise comparison. However, it is not uncommon for these use cases to have some other need for indirection that would also make this work (for instance, in-place sorting by address).

In the SG1 meeting at Issaquah, the first case raised for discussion that takes a `Compare` function object was `sort` ([alg.sort]). However, given that the standard library implementation of this algorithm is in-place, comparison based on the address of the arguments would be nonsensical, given that the objects' addresses would change as the sort progresses. Other needs for preservation of object identity (in the context of `sort`) could not immediately be devised, and we have not come up with any further use cases after some thought. For the non-modifying parallel algorithms that use `Compare` (such as `min_element`, [alg.min.max]), the conceivable use cases for the addresses of the arguments to `Compare` are esoteric enough that the imposition of an indirection requirement is not onerous.

Most of the algorithms that take a `BinaryPredicate` argument are even less problematic than the first two. An example here would be `unique` [alg.unique]. We considered an amendment of the wording in §2 Proposed Wording for the case of `BinaryPredicate` that would require the relationship (in terms of total ordering) between the addresses of the arguments to be the same as in the original objects, but we decided against this in the interest of reduced conceptual overhead.

The least problematic cases are those that take a `UnaryOperation` or `BinaryOperation` function object (note that `for_each` [alg.foreach] is specifically excluded from this, since it takes an argument with the template parameter name `Function` instead). An example of this is `transform` [alg.transform]. Since

these take an argument (or two arguments) and return an object by value (for which the function object has no control over object identity, once copied), it is hard to imagine many use cases where the object identity of the arguments to these function objects is important, since that of the return value cannot be. We do not claim that there are no use cases here affected by the proposed change, but it is a pretty reasonable argument that the potential benefits in terms of implementation flexibility outweigh the inconvenience of requiring indirection in these corner cases.

The note in [§2 Proposed Wording](#) is intended to mimic that of the note in paragraph 10 of [algorithms.general], which references similar interactions with the function objects themselves. In other words, the standard already prohibits the reliance on the object identity of these function objects themselves, so the proposed extension of this prohibition to the function objects' arguments is not unprecedented.

§ 3.1. The "Default Function Object" Clause

One more piece of the proposed wording in [§2 Proposed Wording](#) requires further elaboration. We have inserted the following wording relating to the "default" function objects used by these algorithms:

[...] [and the operators used by the analogous overloads to these parallel algorithms that could be formed by the invocation with the specified default predicate or operation \(where applicable\)](#) [...]

Consider the following code:

```
template <typename T>
void call_a(vector<T>& seq) {
    std::sort(std::par, seq.begin(), seq.end());
}
template <typename T>
void call_b(vector<T>& seq) {
    std::sort(std::par, seq.begin(), seq.end(),
        [&](T const& a, T const& b){ return a < b; }
    );
}
```

From the description of `sort` and other algorithms that take a `Compare` function object ([alg.sorting]), one would expect that `call_a` and `call_b` should have equivalent behavior, regardless of the type of `T`. But consider the following class with a user-defined `operator<()`:

```
struct Counted {
    int value;
    mutable int compares = 0;
    bool operator<(Counted const& other) const {
        ++compares;
        ++other.compares;
        return value < other.value;
    }
}
```

The intent of paragraph 1 of [\[algorithms.parallel.user\]](#) is clearly to prohibit this sort of behavior, but under the current wording, invoking `call_a` with a `vector<Counted>` would be allowed, but invoking `call_b` would not, since it gives a user-defined `Compare` that modifies objects via the arguments. The extension of this prohibition to argument object identity, consistent with the rest of this discussion, also makes sense.

§ 4. Summary

As we see it, the positives and negative of adopting the above wording can be summarized as follows:

Positives:

- Increased implementation flexibility
 - Greater potential for implementation performance would likely lead to wider adoption.
- Better default behavior in future implementations (when such behaviors as the one discussed herein could be subsumed into standard options on an Executor [\[P0443R0\]](#) or [ExecutionPolicy](#))
 - In particular, some members of SG1 were keen to make sure that C++17 code should have reasonable compatibility "by default" with future implementations that may want to copy arguments to memory addresses not accessible from the calling context (e.g., a GPGPU; note that there is currently no concept of inaccessible memory under the memory model of the current standard).

Negatives:

- Increased conceptual overhead
 - This effect is likely negligible since virtually all violating uses proposed after substantial discussion would be corner cases.
- Users who want address-dependent behavior need to implement a wrapper
 - This drawback is made less severe since these users are already providing a custom function object anyway.

§ References

§ Informative References

[P0443R0]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysisen, Carter Edwards. [A Unified Executors Proposal for C++](#). 17 October 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0443r0.html>

[P0488R0]

Barry Hedquist. [WG21 Working paper: NB Comments, ISO/IEC CD 14882](#). 18 October 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0488r0.pdf>